

KRUTIK CYBER EXPERT

PHP COMPLETE COURSE

Server-Side | Database | All Functions

BY KRUTIK 
CYBER EXPERT

"Learn Free. Build Dynamic Web."

CHAPTER 1: PHP INTRODUCTION

What is PHP?

- Hypertext Preprocessor (Recursive acronym)
- Created by Rasmus Lerdorf in 1995
- Server-side scripting language
- Latest Version: PHP 8.4

Why PHP?

- ✓ Widely used (70%+ of websites use PHP)
- ✓ Works with HTML (embedded)
- ✓ Connects to all databases (MySQL, PostgreSQL)
- ✓ Huge ecosystem (Laravel, WordPress)
- ✓ Free and open source
- ✓ Runs on all servers (Apache, Nginx)

What can you do?

- ✓ Dynamic websites
- ✓ E-commerce sites
- ✓ CMS (WordPress, Drupal, Joomla)
- ✓ Web APIs
- ✓ File handling

- ✓ Session management
- ✓ User authentication

SETUP REQUIREMENTS:

1. Web Server (Apache/Nginx)
2. PHP (php.net/download)
3. Database (MySQL/SQLite/PostgreSQL)
4. Or use XAMPP/WAMP/MAMP (All-in-one)

INSTALLATION CHECK:

```
// Create file: info.php
<?php
phpinfo();
?>
```

```
// Open in browser: http://localhost/info.php
```

CHAPTER 2: PHP SYNTAX BASICS

A) PHP TAGS:

CODE:

```
<!-- PHP inside HTML -->
<!DOCTYPE html>
<html>
<body>

<?php
    echo "Hello World!";
?>

</body>
</html>
```

B) OUTPUT TO BROWSER:

CODE:

```
<?php
// echo - output one or more strings
echo "Hello World!";
echo "Hello", " ", "World!"; // Multiple parameters

// print - output one string (slower)
print "Hello World!";
print "Hello World!"; // Only one parameter

// print_r - output array/object
print_r(["apple", "banana", "mango"]);

// var_dump - output with type and length
var_dump(["apple", "banana", "mango"]);

// var_export - valid PHP code output
var_export(["apple", "banana", "mango"]);
?>
```

C) CASE SENSITIVITY:

CODE:

```
<?php
// Variables are case-sensitive
$name = "Krutik";
$Name = "Rahul";
echo $name; // Krutik
echo $Name; // Rahul

// Keywords are NOT case-sensitive
ECHO "Hello"; // Works
EcHo "Hello"; // Works
echo "Hello"; // Works
?>
```

D) COMMENTS:

CODE:

```
<?php
    // This is a single-line comment
    # This is also a single-line comment
    /*
        This is a multi-line comment
        Can span multiple lines
    */

    echo "Hello"; // This is an inline comment
?>
```

CHAPTER 3: VARIABLES AND DATA TYPES

A) VARIABLES (Start with \$):

CODE:

```
<?php
    // Variable declaration
    $name = "Krutik";
    $age = 25;
    $price = 99.99;
    $is_student = true;
    $colors = ["red", "green", "blue"];
    $nothing = null;

    echo $name; // Krutik
?>
```

B) DATA TYPES:

CODE:

```
<?php
    // 1. String
    $name = "Krutik";
    $email = 'krutik@example.com';

    // 2. Integer
```

```

$age = 25;
$negative = -10;

// 3. Float (Double)
$price = 99.99;
$pi = 3.14159;

// 4. Boolean
$is_active = true;
$is_admin = false;

// 5. Array
$fruits = ["apple", "banana", "mango"];
$person = ["name" => "Krutik", "age" => 25];

// 6. Object
$obj = new stdClass();
$obj->name = "Krutik";

// 7. Null
$empty = null;

// 8. Resource (file, database connection)
$file = fopen("file.txt", "r");
?>

```

C) TYPE CHECKING:

CODE:

```

<?php
    $var = "Hello";

    gettype($var);      // "string"
    is_string($var);    // true
    is_int($var);       // false
    is_float($var);     // false
    is_bool($var);      // false
    is_array($var);     // false
    is_null($var);      // false
    is_object($var);    // false
    is_numeric("123");   // true
    is_numeric("abc");   // false
    isset($var);        // true
    empty($var);        // false (empty string)
?>

```

D) TYPE CONVERSION:

CODE:

```
<?php
    // String to Integer
    intval("123");    // 123
    intval("12.5");    // 12
    intval("abc");    // 0

    // String to Float
    floatval("12.5");  // 12.5
    floatval("abc");   // 0

    // To String
    strval(123);       // "123"
    strval(12.5);      // "12.5"

    // Implicit conversion
    $num = "123" + 10; // 133 (string converted to int)
    $str = 123 . "abc"; // "123abc" (concatenation)
?>
```

CHAPTER 4: OPERATORS

A) ARITHMETIC OPERATORS:

CODE:

```
<?php
    $a = 10;
    $b = 3;

    echo $a + $b; // 13
    echo $a - $b; // 7
    echo $a * $b; // 30
    echo $a / $b; // 3.333
    echo $a % $b; // 1
    echo $a ** $b; // 1000
```

?>

B) ASSIGNMENT OPERATORS:

CODE:

```
<?php
    $x = 10;
    $x += 5; // $x = 15
    $x -= 3; // $x = 12
    $x *= 2; // $x = 24
    $x /= 4; // $x = 6
    $x %= 4; // $x = 2
    $x .= "abc"; // $x = "2abc" (string concatenation)
?>
```

C) COMPARISON OPERATORS:

CODE:

```
<?php
    $a = 5;
    $b = "5";

    var_dump($a == $b); // true (value equal)
    var_dump($a === $b); // false (value and type equal)
    var_dump($a != $b); // false
    var_dump($a !== $b); // true
    var_dump($a > $b); // false
    var_dump($a < $b); // false
    var_dump($a >= $b); // true
    var_dump($a <= $b); // true

    // Spaceship (PHP 7+)
    echo 5 <=> 10; // -1 (less than)
    echo 10 <=> 10; // 0 (equal)
    echo 15 <=> 10; // 1 (greater than)
?>
```

D) LOGICAL OPERATORS:

CODE:

```

<?php
    $a = true;
    $b = false;

    var_dump($a && $b); // false (AND)
    var_dump($a and $b); // false (and)
    var_dump($a || $b); // true (OR)
    var_dump($a or $b); // true (or)
    var_dump(!$a);      // false (NOT)

    // Null coalescing (PHP 7+)
    $name = $_GET['name'] ?? 'Guest'; // Use 'Guest' if not set
    $name = $_GET['name'] ?? $_POST['name'] ?? 'Guest';

    // Null coalescing assignment (PHP 7.4+)
    $data = null;
    $data ??= 'default'; // $data = 'default'
?>

```

E) INCREMENT/DECREMENT:

CODE:

```

<?php
    $x = 5;
    echo $x++; // 5 (then $x becomes 6)
    echo ++$x; // 7 ($x becomes 7 then output)
    echo $x--; // 7 (then $x becomes 6)
    echo --$x; // 5 ($x becomes 5 then output)
?>

```

CHAPTER 5: STRINGS

A) STRING CREATION:

CODE:

```

<?php
    // Single quotes (literal)

```



```

$name = 'Krutik';
echo 'Hello $name'; // Hello $name (no variable parsing)

// Double quotes (parses variables)
$name = 'Krutik';
echo "Hello $name"; // Hello Krutik
echo "Hello {$name}"; // Hello Krutik (curly braces)

// Heredoc (multi-line)
$html = <<<HTML
    <div>
        <h1>Hello $name</h1>
        <p>This is multi-line.</p>
    </div>
HTML;

// Nowdoc (multi-line, no parsing)
$html = <<<'HTML'
    <div>
        <h1>Hello $name</h1> <!-- $name not parsed -->
    </div>
HTML;
?>

```

B) STRING FUNCTIONS:

CODE:

```

<?php
    $str = "Hello World";

    // Length
    strlen($str);    // 11

    // Case
    strtoupper($str); // "HELLO WORLD"
    strtolower($str); // "hello world"
    ucfirst($str);    // "Hello world"
    ucwords($str);    // "Hello World"

    // Trim
    trim(" Hello "); // "Hello"
    ltrim(" Hello "); // "Hello "
    rtrim(" Hello "); // " Hello"

    // Substring

```

```
substr($str, 0, 5); // "Hello"
substr($str, 6);    // "World"
substr($str, -5);   // "World"
```

```
// Find
strpos($str, "World"); // 6
strpos($str, "x");     // false
strrpos($str, "l");    // 9 (last occurrence)
```

```
// Replace
str_replace("World", "PHP", $str); // "Hello PHP"
str_replace(["H", "W"], ["J", "X"], $str); // "Jello Xorld"
```

```
// Split/Join
explode(" ", $str); // ["Hello", "World"]
implode("-", ["Hello", "World"]); // "Hello-World"
```

```
// Check
str_contains($str, "World"); // true (PHP 8+)
str_starts_with($str, "Hello"); // true (PHP 8+)
str_ends_with($str, "World"); // true (PHP 8+)
```

```
// Escape HTML
htmlspecialchars("<script>alert('xss')</script>");
```

```
// Strip HTML tags
strip_tags("<p>Hello</p>"); // "Hello"
```

?>

CHAPTER 6: NUMBERS AND MATH

A) NUMBER FUNCTIONS:

CODE:

```
<?php
$num = 123.456;

// Rounding
round($num);      // 123
round($num, 2);   // 123.46
ceil($num);       // 124
```

```

floor($num);      // 123

// Check
is_numeric("123"); // true
is_int(123);       // true
is_float(12.5);    // true
is_finite(100);    // true
is_infinite(1/0);  // true
is_nan(sqrt(-1));  // true

// Convert
intval(12.5);      // 12
floatval("12.5");  // 12.5

// Random
rand();            // Random integer
rand(1, 10);       // 1-10
mt_rand(1, 10);    // Better random

// Format
number_format(1234.56, 2, '.', ''); // "1,234.56"
?>

```

B) MATH FUNCTIONS:

CODE:

```

<?php
// Basic
abs(-5);      // 5
min(1, 5, 3, 9); // 1
max(1, 5, 3, 9); // 9
pow(2, 3);    // 8
sqrt(16);     // 4

// Trigonometry
sin(0);       // 0
cos(0);       // 1
tan(0);       // 0

// Constants
M_PI;         // 3.14159
M_E;          // 2.71828
?>

```

CHAPTER 7: ARRAYS

A) CREATING ARRAYS:

CODE:

```
<?php
// Indexed array
$fruits = ["apple", "banana", "mango"];
$fruits = array("apple", "banana", "mango");

// Associative array (key-value)
$person = [
    "name" => "Krutik",
    "age" => 25,
    "city" => "Mumbai"
];

// Multi-dimensional
$matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];
?>
```

B) ACCESSING ARRAYS:

CODE:

```
<?php
$fruits = ["apple", "banana", "mango"];
echo $fruits[0];    // "apple"
echo $fruits[1];    // "banana"

$person = ["name" => "Krutik", "age" => 25];
echo $person["name"]; // "Krutik"
echo $person["age"];  // 25

// Length
```

```
count($fruits);    // 3
?>
```

C) ARRAY FUNCTIONS:

CODE:

```
<?php
$arr = ["apple", "banana", "mango"];

// Add/Remove
array_push($arr, "orange"); // Add to end
array_pop($arr);           // Remove from end
array_unshift($arr, "grape"); // Add to start
array_shift($arr);         // Remove from start

// Merge
$arr1 = ["a", "b"];
$arr2 = ["c", "d"];
$merged = array_merge($arr1, $arr2); // ["a","b","c","d"]

// Combine (keys + values)
$keys = ["name", "age"];
$values = ["Krutik", 25];
$person = array_combine($keys, $values); // ["name"=>"Krutik","age"=>25]

// Search
in_array("apple", $arr); // true
array_search("banana", $arr); // 1 (index)
array_key_exists("name", $person); // true

// Sort
sort($arr);           // Ascending
rsort($arr);          // Descending
asort($person);       // Ascending (preserve keys)
ksort($person);       // Sort by keys
krsort($person);      // Sort by keys (reverse)

// Unique
$num = [1,2,2,3,3,4];
array_unique($num);    // [1,2,3,4]

// Reverse
array_reverse($arr);   // Reverse order

// Slice
```

```

array_slice($arr, 1, 2); // Start at 1, length 2

// Splice (modify)
array_splice($arr, 1, 1); // Remove 1 at index 1
array_splice($arr, 1, 0, ["new"]); // Insert at index 1

// Extract keys/values
array_keys($person); // ["name", "age"]
array_values($person); // ["Krutik", 25]

// Filter
$numbers = [1,2,3,4,5];
$evens = array_filter($numbers, function($n) {
    return $n % 2 == 0;
}); // [2,4]

// Map
$doubled = array_map(function($n) {
    return $n * 2;
}, $numbers); // [2,4,6,8,10]

// Reduce
$sum = array_reduce($numbers, function($acc, $curr) {
    return $acc + $curr;
}, 0); // 15
?>

```

D) LOOPING ARRAYS:

CODE:

```

<?php
$fruits = ["apple", "banana", "mango"];

// For loop
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i];
}

// Foreach
foreach ($fruits as $fruit) {
    echo $fruit;
}

// Foreach with key
$person = ["name" => "Krutik", "age" => 25];

```

```
    foreach ($person as $key => $value) {  
        echo "$key: $value";  
    }  
?>
```

CHAPTER 8: CONDITIONALS

A) IF-ELSE:

CODE:

```
<?php  
    $age = 18;  
  
    if ($age >= 18) {  
        echo "You can vote";  
    } else {  
        echo "You cannot vote";  
    }  
  
    // Elseif  
    $marks = 85;  
    if ($marks >= 90) {  
        echo "A+";  
    } elseif ($marks >= 80) {  
        echo "A";  
    } elseif ($marks >= 70) {  
        echo "B";  
    } else {  
        echo "C";  
    }  
?>
```

B) SWITCH:

CODE:

```
<?php  
    $day = "Monday";
```

```
switch ($day) {
    case "Monday":
        echo "Start of week";
        break;
    case "Friday":
        echo "Almost weekend";
        break;
    case "Saturday":
    case "Sunday":
        echo "Weekend!";
        break;
    default:
        echo "Midweek";
}
?>
```

C) MATCH (PHP 8+):

CODE:

```
<?php
$status = 404;

$message = match($status) {
    200 => "OK",
    404 => "Not Found",
    500 => "Server Error",
    default => "Unknown",
};
echo $message; // "Not Found"

// Match with conditions
$age = 25;
$result = match(true) {
    $age < 18 => "Minor",
    $age >= 18 && $age < 60 => "Adult",
    $age >= 60 => "Senior",
};
?>
```

D) TERNARY OPERATOR:

CODE:

```
<?php
    $age = 18;
    $status = ($age >= 18) ? "Adult" : "Minor";
    echo $status; // "Adult"

    // Null coalescing operator (??)
    $name = $_GET['name'] ?? 'Guest';

    // Null coalescing assignment (??=)
    $name ??= 'Guest'; // Set if null
?>
```

CHAPTER 9: LOOPS

A) FOR LOOP:

CODE:

```
<?php
    for ($i = 0; $i < 5; $i++) {
        echo $i; // 0,1,2,3,4
    }

    // Nested loop
    for ($i = 1; $i <= 3; $i++) {
        for ($j = 1; $j <= 3; $j++) {
            echo "$i x $j = " . ($i * $j) . "\n";
        }
    }
?>
```

B) WHILE LOOP:

CODE:

```
<?php
    $i = 0;
    while ($i < 5) {
        echo $i;
        $i++;
    }
```

```
}  
?>
```

C) DO-WHILE LOOP:

CODE:

```
<?php  
    $i = 0;  
    do {  
        echo $i;  
        $i++;  
    } while ($i < 5);  
?>
```

D) FOREACH LOOP:

CODE:

```
<?php  
    $fruits = ["apple", "banana", "mango"];  
  
    foreach ($fruits as $fruit) {  
        echo $fruit;  
    }  
  
    // With key  
    $person = ["name" => "Krutik", "age" => 25];  
    foreach ($person as $key => $value) {  
        echo "$key: $value";  
    }  
?>
```

E) BREAK AND CONTINUE:

CODE:

```
<?php  
    // Break - exit loop  
    for ($i = 0; $i < 10; $i++) {  
        if ($i == 5) break;  
        echo $i; // 0,1,2,3,4  
    }  
?>
```

```
}

// Continue - skip iteration
for ($i = 0; $i < 10; $i++) {
    if ($i % 2 == 0) continue;
    echo $i; // 1,3,5,7,9
}
?>
```

CHAPTER 10: FUNCTIONS

A) FUNCTION DECLARATION:

CODE:

```
<?php
// Basic function
function greet() {
    echo "Hello!";
}
greet(); // "Hello!"

// With parameters
function greetUser($name) {
    echo "Hello $name!";
}
greetUser("Krutik"); // "Hello Krutik!"

// With return value
function add($a, $b) {
    return $a + $b;
}
$result = add(5, 3); // 8
?>
```

B) DEFAULT PARAMETERS:

CODE:

```
<?php
```

```
function greet($name = "Guest") {  
    return "Hello $name!";  
}  
echo greet();      // "Hello Guest!"  
echo greet("Krutik"); // "Hello Krutik!"  
?>
```

C) TYPE HINTING (PHP 7+):

CODE:

```
<?php  
// Parameter types  
function add(int $a, int $b): int {  
    return $a + $b;  
}  
echo add(5, 3);    // 8  
// echo add("5", "3"); // Error  
  
// Return type  
function greet(string $name): string {  
    return "Hello $name!";  
}  
?>
```

D) VARIABLE NUMBER OF ARGUMENTS:

CODE:

```
<?php  
// Using func_get_args()  
function sum() {  
    $total = 0;  
    foreach (func_get_args() as $num) {  
        $total += $num;  
    }  
    return $total;  
}  
echo sum(1,2,3,4,5); // 15  
  
// Using ... (spread operator)  
function sum2(...$numbers) {  
    return array_sum($numbers);  
}
```

```
    echo sum2(1,2,3,4,5); // 15
?>
```

E) ANONYMOUS FUNCTIONS (Closures):

CODE:

```
<?php
// Anonymous function
$greet = function($name) {
    return "Hello $name!";
};
echo $greet("Krutik"); // "Hello Krutik!"

// Arrow function (PHP 7.4+)
$add = fn($a, $b) => $a + $b;
echo $add(5, 3); // 8
?>
```

F) VARIABLE SCOPE:

CODE:

```
<?php
$global = "I'm global";

function myFunc() {
    $local = "I'm local";
    // echo $global; // ❌ Error - not accessible

    // Use global
    global $global;
    echo $global;    // "I'm global"

    // Or use $GLOBALS
    echo $GLOBALS['global']; // "I'm global"
}

// Static variables (persist between calls)
function counter() {
    static $count = 0;
    $count++;
    return $count;
}
```

```
echo counter(); // 1
echo counter(); // 2
echo counter(); // 3
?>
```

CHAPTER 11: SUPERGLOBALS (Predefined Variables)

A) COMMON SUPERGLOBALS:

Variable	Description
\$_GET	URL parameters (GET request)
\$_POST	Form data (POST request)
\$_REQUEST	GET, POST, COOKIE combined
\$_SERVER	Server information
\$_SESSION	Session variables
\$_COOKIE	Cookie variables
\$_FILES	File uploads
\$_ENV	Environment variables
\$GLOBALS	All global variables

B) \$_GET (URL Parameters):

CODE:

```
// URL: example.com?name=Krutik&age=25
```

```
<?php
echo $_GET['name']; // "Krutik"
echo $_GET['age']; // 25

// With default
$name = $_GET['name'] ?? 'Guest';
?>
```

C) \$_POST (Form Data):

CODE:

```
<!-- HTML Form -->
<form method="POST" action="process.php">
  <input type="text" name="name">
  <input type="submit">
</form>

<!-- process.php -->
<?php
  if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'] ?? "";
    echo "Hello $name!";
  }
?>
```

D) \$_SERVER (Server Info):

CODE:

```
<?php
  echo $_SERVER['HTTP_HOST'];    // localhost
  echo $_SERVER['REQUEST_URI'];  // /page.php?id=1
  echo $_SERVER['REQUEST_METHOD']; // GET/POST
  echo $_SERVER['REMOTE_ADDR'];  // Client IP
  echo $_SERVER['SERVER_NAME'];  // Server name
  echo $_SERVER['DOCUMENT_ROOT']; // Root directory
  echo $_SERVER['PHP_SELF'];      // /path/to/file.php
  echo $_SERVER['HTTP_USER_AGENT']; // Browser info
?>
```

CHAPTER 12: FILE HANDLING

A) FILE FUNCTIONS:

CODE:

```
<?php
```

```

// Read file
$content = file_get_contents("file.txt");
echo $content;

// Write file (overwrite)
file_put_contents("file.txt", "Hello World!");

// Append
file_put_contents("file.txt", "\nNew line", FILE_APPEND);

// Read line by line
$lines = file("file.txt"); // Array of lines
foreach ($lines as $line) {
    echo $line;
}

// Check if file exists
if (file_exists("file.txt")) {
    echo "File exists";
}

// Get file info
filesize("file.txt"); // Size in bytes
filemtime("file.txt"); // Last modified time
is_readable("file.txt"); // true/false
is_writable("file.txt"); // true/false
?>

```

B) FILE OPERATIONS (fopen, fread, fwrite):

CODE:

```

<?php
// Open file
$file = fopen("file.txt", "r"); // r=read, w=write, a=append

// Read
$content = fread($file, filesize("file.txt"));

// Read line by line
while (($line = fgets($file)) !== false) {
    echo $line;
}

// Write
$file = fopen("file.txt", "w");

```



```
fwrite($file, "Hello World!");  
fclose($file);
```

```
// Append  
$file = fopen("file.txt", "a");  
fwrite($file, "\nNew line");  
fclose($file);  
?>
```

C) DIRECTORY FUNCTIONS:

CODE:

```
<?php  
// Create directory  
mkdir("new_folder");  
  
// List files  
$files = scandir(".");  
foreach ($files as $file) {  
    echo $file;  
}  
  
// Check if directory  
is_dir("folder");  
  
// Delete directory (must be empty)  
rmdir("empty_folder");  
  
// Delete file  
unlink("file.txt");  
?>
```

CHAPTER 13: SESSIONS AND COOKIES

A) COOKIES:

CODE:

```
<?php
```

```

// Set cookie (expires in 1 hour)
setcookie("user", "Krutik", time() + 3600);

// Set cookie with path
setcookie("user", "Krutik", time() + 3600, "/");

// Read cookie
$user = $_COOKIE['user'] ?? 'Guest';
echo $user; // "Krutik"

// Delete cookie
setcookie("user", "", time() - 3600);
?>

```

B) SESSIONS:

CODE:

```

<?php
// Start session (must be before any output)
session_start();

// Set session variable
$_SESSION['user'] = "Krutik";
$_SESSION['user_id'] = 123;

// Read session
$user = $_SESSION['user'] ?? 'Guest';
echo $user; // "Krutik"

// Check if logged in
if (isset($_SESSION['user'])) {
    echo "Logged in as " . $_SESSION['user'];
} else {
    echo "Please login";
}

// Destroy session
session_destroy();
session_unset(); // Clear all session variables
?>

```

C) LOGIN SYSTEM EXAMPLE:

CODE:

```
<!-- login.php -->
<?php
session_start();

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    // Check database (simplified)
    if ($username == 'admin' && $password == 'password') {
        $_SESSION['user'] = $username;
        $_SESSION['logged_in'] = true;
        header('Location: dashboard.php');
        exit();
    } else {
        $error = "Invalid credentials";
    }
}
?>

<form method="POST">
    <input type="text" name="username" placeholder="Username">
    <input type="password" name="password" placeholder="Password">
    <button type="submit">Login</button>
    <?php if (isset($error)) echo "<p>$error</p>"; ?>
</form>

<!-- dashboard.php -->
<?php
session_start();

if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {
    header('Location: login.php');
    exit();
}

echo "Welcome " . $_SESSION['user'] . "!";
?>
```

CHAPTER 14: DATABASE (MySQL with PDO)

A) CONNECTING TO DATABASE:

CODE:

```
<?php
// Using PDO (Recommended)
$host = 'localhost';
$dbname = 'test_db';
$username = 'root';
$password = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username,
$password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
} catch(PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

B) SELECT QUERY:

CODE:

```
<?php
try {
    // Prepare statement
    $stmt = $pdo->prepare("SELECT * FROM users");
    $stmt->execute();

    // Fetch all records
    $users = $stmt->fetchAll(PDO::FETCH_ASSOC);
    foreach ($users as $user) {
        echo $user['name'] . " - " . $user['email'];
    }

    // Fetch single record
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    // With WHERE clause (using parameters)
    $stmt = $pdo->prepare("SELECT * FROM users WHERE id = :id");
    $stmt->execute(['id' => 1]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);
```

```

        // Or using ? placeholder
        $stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
        $stmt->execute([1]);
        $user = $stmt->fetch();
    } catch(PDOException $e) {
        echo "Error: " . $e->getMessage();
    }
?>

```

C) INSERT QUERY:

CODE:

```

<?php
    try {
        $name = "Krutik";
        $email = "krutik@example.com";

        // Using named parameters
        $stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (:name,
:email)");
        $stmt->execute([
            ':name' => $name,
            ':email' => $email
        ]);

        // Or using ? placeholders
        $stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (?, ?)");
        $stmt->execute([$name, $email]);

        // Get last inserted ID
        $lastId = $pdo->lastInsertId();
        echo "Inserted user with ID: $lastId";
    } catch(PDOException $e) {
        echo "Error: " . $e->getMessage();
    }
?>

```

D) UPDATE QUERY:

CODE:

```

<?php

```

```

try {
    $id = 1;
    $email = "new@example.com";

    $stmt = $pdo->prepare("UPDATE users SET email = :email WHERE id = :id");
    $stmt->execute([
        ':email' => $email,
        ':id' => $id
    ]);

    // Get affected rows
    $count = $stmt->rowCount();
    echo "Updated $count rows";
} catch(PDOException $e) {
    echo "Error: " . $e->getMessage();
}
?>

```

E) DELETE QUERY:

CODE:

```

<?php
try {
    $id = 1;

    $stmt = $pdo->prepare("DELETE FROM users WHERE id = :id");
    $stmt->execute([':id' => $id]);

    $count = $stmt->rowCount();
    echo "Deleted $count rows";
} catch(PDOException $e) {
    echo "Error: " . $e->getMessage();
}
?>

```

F) COMPLETE CRUD EXAMPLE:

CODE:

```

<?php
// config.php - Database configuration
$host = 'localhost';
$dbname = 'test_db';

```

```

$username = 'root';
$password = "";

$pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// create.php - Insert user
function createUser($name, $email) {
    global $pdo;
    $stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (:name,
:email)");
    return $stmt->execute([':name' => $name, ':email' => $email]);
}

// read.php - Get users
function getUsers() {
    global $pdo;
    $stmt = $pdo->query("SELECT * FROM users");
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

// read.php - Get single user
function getUser($id) {
    global $pdo;
    $stmt = $pdo->prepare("SELECT * FROM users WHERE id = :id");
    $stmt->execute([':id' => $id]);
    return $stmt->fetch(PDO::FETCH_ASSOC);
}

// update.php - Update user
function updateUser($id, $name, $email) {
    global $pdo;
    $stmt = $pdo->prepare("UPDATE users SET name = :name, email = :email WHERE
id = :id");
    return $stmt->execute([':name' => $name, ':email' => $email, ':id' => $id]);
}

// delete.php - Delete user
function deleteUser($id) {
    global $pdo;
    $stmt = $pdo->prepare("DELETE FROM users WHERE id = :id");
    return $stmt->execute([':id' => $id]);
}

// Usage
createUser("Krutik", "krutik@example.com");
$users = getUsers();
foreach ($users as $user) {

```

```
    echo $user['name'] . " - " . $user['email'] . "\n";
}
?>
```

CHAPTER 15: ERROR HANDLING

A) TRY-CATCH (Exceptions):

CODE:

```
<?php
try {
    $number = 10;
    if ($number == 0) {
        throw new Exception("Cannot divide by zero!");
    }
    $result = 10 / $number;
    echo $result;
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
} finally {
    echo "This always runs";
}
?>
```

B) CUSTOM EXCEPTIONS:

CODE:

```
<?php
class ValidationException extends Exception {}
class DatabaseException extends Exception {}

function validateEmail($email) {
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        throw new ValidationException("Invalid email: $email");
    }
    return true;
}
```



```
try {
    validateEmail("invalid");
} catch (ValidationException $e) {
    echo $e->getMessage();
} catch (Exception $e) {
    echo "General error";
}
?>
```

C) ERROR REPORTING LEVELS:

CODE:

```
<?php
// Turn on all errors
error_reporting(E_ALL);

// Turn off notices
error_reporting(E_ALL & ~E_NOTICE);

// Log errors instead of display
ini_set('display_errors', 0);
ini_set('log_errors', 1);

// Custom error handler
function customError($errno, $errstr) {
    echo "<b>Error:</b> [$errno] $errstr";
    error_log("Error: $errstr", 0);
}
set_error_handler("customError");
?>
```

CHAPTER 16: COMPLETE PROJECT - CONTACT FORM WITH DATABASE

HTML (contact.php):

```
<!DOCTYPE html>
<html>
<head>
    <title>Contact Us</title>
```

```

<style>
    body { font-family: Arial; max-width: 600px; margin: 50px auto; padding: 20px; }
    input, textarea { width: 100%; padding: 10px; margin: 10px 0; border: 1px solid
#ddd; border-radius: 5px; }
    button { background: #28a745; color: white; padding: 12px 30px; border: none;
border-radius: 5px; cursor: pointer; }
    .error { color: red; }
    .success { color: green; }
    table { width: 100%; border-collapse: collapse; }
    th, td { padding: 10px; text-align: left; border-bottom: 1px solid #ddd; }
</style>
</head>
<body>
    <h1>Contact Us</h1>

    <?php if (isset($success)): ?>
        <div class="success"><?= $success ?></div>
    <?php endif; ?>
    <?php if (isset($error)): ?>
        <div class="error"><?= $error ?></div>
    <?php endif; ?>

    <form method="POST">
        <input type="text" name="name" placeholder="Your Name" required>
        <input type="email" name="email" placeholder="Your Email" required>
        <textarea name="message" rows="5" placeholder="Your Message"
required></textarea>
        <button type="submit" name="submit">Send Message</button>
    </form>

    <hr>
    <h2>Messages</h2>
    <table>
        <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Message</th>
            <th>Date</th>
        </tr>
        <?php foreach ($messages as $msg): ?>
            <tr>
                <td><?= htmlspecialchars($msg['name']) ?></td>
                <td><?= htmlspecialchars($msg['email']) ?></td>
                <td><?= htmlspecialchars($msg['message']) ?></td>
                <td><?= $msg['created_at'] ?></td>
            </tr>
        <?php endforeach; ?>
    </table>

```

```
</body>
</html>
```

PHP (contact.php - Backend):

```
<?php
// Database connection
try {
    $pdo = new PDO("mysql:host=localhost;dbname=test_db", "root", "");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}

// Handle form submission
$success = $error = null;
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['submit'])) {
    $name = trim($_POST['name'] ?? "");
    $email = trim($_POST['email'] ?? "");
    $message = trim($_POST['message'] ?? "");

    // Validate
    if (empty($name) || empty($email) || empty($message)) {
        $error = "All fields are required!";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = "Invalid email address!";
    } else {
        try {
            $stmt = $pdo->prepare("INSERT INTO messages (name, email, message)
VALUES (:name, :email, :message)");
            $stmt->execute([
                ':name' => $name,
                ':email' => $email,
                ':message' => $message
            ]);
            $success = "Message sent successfully!";
        } catch (PDOException $e) {
            $error = "Error: " . $e->getMessage();
        }
    }
}

// Fetch all messages
try {
    $stmt = $pdo->query("SELECT * FROM messages ORDER BY created_at DESC");
    $messages = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
} catch (PDOException $e) {  
    $messages = [];  
}
```

```
// Include HTML  
include 'contact_form.html';  
?>
```

```
<!-- Database Table (SQL) -->  
CREATE TABLE messages (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    message TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

CHAPTER 17: QUICK REFERENCE - ALL FUNCTIONS

STRING FUNCTIONS:

`strlen()` `strtoupper()` `strtolower()` `ucfirst()` `ucwords()` `trim()`
`ltrim()` `rtrim()` `substr()` `strpos()` `strrpos()` `str_replace()`
`str_ireplace()` `explode()` `implode()` `str_contains()` `str_starts_with()`
`str_ends_with()` `htmlspecialchars()` `strip_tags()` `addslashes()`
`stripslashes()` `htmlentities()` `nl2br()`

ARRAY FUNCTIONS:

`count()` `array_push()` `array_pop()` `array_shift()` `array_unshift()`
`array_merge()` `array_combine()` `in_array()` `array_search()`
`array_key_exists()` `sort()` `rsort()` `asort()` `ksort()` `krsort()`
`array_reverse()` `array_unique()` `array_slice()` `array_splice()`
`array_keys()` `array_values()` `array_filter()` `array_map()`
`array_reduce()` `array_diff()` `array_intersect()` `array_sum()`
`array_product()` `array_chunk()` `array_rand()`

FILE FUNCTIONS:

`file_get_contents()` `file_put_contents()` `file()` `fopen()` `fread()`
`fwrite()` `fclose()` `fgets()` `file_exists()` `filesize()`
`filemtime()` `is_readable()` `is_writable()` `mkdir()` `scandir()`
`unlink()` `rmdir()` `rename()` `copy()`

MATH FUNCTIONS:

`abs()` `round()` `ceil()` `floor()` `min()` `max()` `pow()` `sqrt()` `rand()`
`mt_rand()` `number_format()` `is_numeric()` `is_int()` `is_float()`

DATE FUNCTIONS:

**date() time() strtotime() date_default_timezone_set()
date_create() date_format() date_diff()**

DATABASE (PDO):

**prepare() execute() fetch() fetchAll() fetchColumn()
rowCount() lastInsertId() setAttribute()**

SUPERGLOBALS:

**\$_GET \$_POST \$_REQUEST \$_SERVER \$_SESSION \$_COOKIE
\$_FILES \$_ENV \$GLOBALS**

CERTIFICATE OF COMPLETION

This certifies that the reader has completed

PHP COMPLETE COURSE

By

KRUTIK  CYBER EXPERT

Topics Covered:

- ✓ **PHP Introduction (Setup & Tags)**
- ✓ **Variables and Data Types**
- ✓ **Operators (Arithmetic, Comparison, Logical)**
- ✓ **Strings (All Functions)**
- ✓ **Numbers and Math**
- ✓ **Arrays (All Functions)**
- ✓ **Conditionals (If-else, Switch, Match)**
- ✓ **Loops (For, While, Foreach)**
- ✓ **Functions (Parameters, Return, Type Hinting)**
- ✓ **Superglobals (\$_GET, \$_POST, \$_SERVER)**
- ✓ **File Handling (Read, Write, Delete)**
- ✓ **Sessions and Cookies (Login System)**
- ✓ **Database (PDO - CRUD Operations)**
- ✓ **Error Handling (Try-Catch, Custom Exceptions)**
- ✓ **Complete Project (Contact Form with Database)**
- ✓ **Quick Reference (All Functions)**

"Build Dynamic Web Apps."

**NOTE: This PDF is completely FREE. If you paid for it, you got scammed.
Share it with everyone who wants to learn PHP! 🚀**

